

M1.1: Lexer & Parser Syntax Resolution

Theory: Parses DSL code into Abstract Syntax Trees (AST) using flex/bison LALR(1) grammars.

Details: bpftrace splits scripts into token streams containing probe definitions, filters, and action blocks, mapping them into structured AST expressions.

Trade-off: The custom DSL avoids full Clang frontend memory bloat, but lacks complex C preprocessor warning hints during syntax check errors.

M1.2: Semantic Analysis & Type Verification

Theory: Validates types, resolves symbols, and ensures Map key-value monomorphism.

Details: 'SemanticAnalyser' visits AST nodes, aligning context offsets with BTF struct layouts and verifying that variables maintain uniform types across execution scopes.

Trade-off: Type analysis must run in userland; semantic errors must trigger compile-time aborts to prevent the kernel Verifier from rejecting bytecode.

M1.3: AST to LLVM IR Translation

Theory: Maps AST nodes to target LLVM IR representations.

Details: 'CodegenLLVM' walks validated AST structures, utilizing LLVM C++ APIs to generate platform-agnostic SSA LLVM IR representing Map updates or function calls.

Trade-off: Leveraging LLVM IR enables rich optimizations but increases binary size due to the bundled LLVM JIT engine.

M1.4: Variable Scopes & Storage Models

Theory: Local stack variable registers and persistent global/thread-local BPF Maps.

Details: Supports local variables ('\$x') in BPF stack frames, global variables('@x') in global BPF maps, and thread-local variables('@usdt_tid') in tid-indexed maps.

Trade-off: Lacking static heap space in eBPF, all shared variables are stored in maps, introducing tiny search lookups.

M1.5: Builtin Variables & Aggregation Operators

Theory: In-kernel fast aggregation via count, sum, and hist maps helpers.

Details: Built-in variables (e.g. 'pid', 'comm') map to kernel helpers, while operators (e.g. 'count()', 'hist()') update maps natively using atomic add operations.

Trade-off: Compacting data inside the kernel reduces serialization costs and limits userland CPU usage.

M1.6: BTF Kernels Structural Offset Deduction

Theory: Decodes struct layouts using BTF descriptors.

Details: Parses system BTF metadata to calculate target field offsets (e.g. 'task_struct->state'), compiling them into safe 'bpf_probe_read' instruction sequences.

Trade-off: Requires '/sys/kernel/btf/vmlinux' support; without it, bpftrace falls back to manually parsing header paths.

M2.1: LLVM MCJIT Compiler Backend

Theory: Emits eBPF ELF binaries from LLVM IR.

Details: Invokes LLVM target code generators to convert IR into eBPF binaries, resolving map file descriptor references in relocation sections.

Trade-off: JIT compiling on-the-fly ensures machine compatibility, but consumes minor compilation memory on low-resource hardware.

M2.2: BPF Map Allocation & Strategies

Theory: Allocates thread-safe per-CPU hashes and arrays to bypass locks.

Details: Allocates Map slots via 'bpf_create_map', utilizing 'BPF_MAP_TYPE_PERCPU_HASH' to allocate independent memory regions per CPU core.

Trade-off: Per-CPU allocations eliminate locking overhead but require additional CPU resources when merging data in userland.

M2.3: Probe Attachment & Hooking Mechanics

Theory: Modifies entry instructions for dynamic hooking.

Details: Dynamic probes (kprobes) modify instruction headers to jump to handler stubs, tracepoints hook into static kernel anchors, and USDT writes software traps into target process memory.

Trade-off: Dynamic kprobes require updates if kernel symbols change; USDT instrumentation adds tiny page-write pauses during attach.

M2.4: Ring Buffer Communication Channel

Theory: High-speed asynchronous ring buffer data delivery.

Details: Injects trace records into userland using shared memory pools ('ring_buffer' or 'perf_event_array') queried by user-space epoll threads.

Trade-off: High throughput design; if user-space consumption lags behind kernel generation, the queue fills up and events are dropped.

M2.5: Multi-Probe Coordination & Session Binding

Theory: Aggregates multiple hooks in a single tracking Session.

Details: Binds multiple probes under one shared 'BpfContext', allowing them to read and write to the same Map file descriptors.

Trade-off: Guarantees atomic consistency across metrics but rejects the entire script if a single probe fails the Verifier check.

M3.2: Session Lifecycle & Detach Cleanup

Theory: Graceful termination and cleanup of kernel hooks.

Details: Triggers cleanup upon SIGINT or calling 'exit()', unlinking probes via 'ioctl(UNREGISTER)' and closing Map descriptors.

Trade-off: If bpftrace is killed abruptly (e.g. 'kill -9'), hooks remain registered in the kernel and must be cleared manually via debugfs.

M3.6: USDT Semaphore Instrumentation

Theory: Controls runtime instrumentation using Stap USDT semaphore offsets.

Details: Resolves semaphores in target ELF notes, writing to memory offsets to activate tracing code.

Trade-off: Modifying target process memory requires elevated host permissions ('CAP_SYS_PTRACE').

M4.2: Resource Boundaries

Theory: Protects host resources using static limitations.

Details: Limits Map keys to 5120 entries and restricts the kernel stack to 512 bytes to prevent out-of-memory issues.

Trade-off: Prevents resource exhaustion but can result in data loss if metrics exceed limits.

M2.6: Verifier Static Analysis Security

Theory: Verifies safety constraints before executing bytecode in kernel space.

Details: The kernel Verifier runs DFS checks on bytecode to ensure there are no infinite loops, invalid stack offsets, or uninitialized memory accesses.

Trade-off: Guarantees kernel stability but restricts control flow complexity and maximum instruction count.

M3.4: Stack Backtrace & Symbol Translation

Theory: Translates instruction pointers into human-readable symbols.

Details: Captures frame registers in-kernel, then decodes IPs in user-space using '/proc/kallsyms' (kernel) or DWARF mapping tables (user-space).

Trade-off: Resolving symbols (especially C++ demangling) is CPU-heavy; utilize caching to mitigate performance impact.

M3.7: Filters Generation

Theory: Checks filter matches and aborts early in kernel space.

Details: Compiles filters (e.g. '/pid == 10/') to register comparison instructions that abort early if unmatched.

Trade-off: Minimizes transfer overhead but adds branch evaluation costs to every probe trigger.

M4.3: Kernel Adaptation

Theory: Detects kernel features and adjusts JIT strategies dynamically.

Details: Probes kernel capabilities at startup, testing for 'ring_buffer' support and helper availability.

Trade-off: Maximizes script portability but adds minor startup feature detection delay.

M4.4: Kernel Stack Limit Protection

Theory: Enforces the eBPF 512-byte stack frame limit.

Details: Flags local arrays exceeding 512 bytes during compilation, suggesting Per-CPU maps instead.

Trade-off: Prevents stack overflows but requires alternative scratch map implementations.

M3.1: User-Space Map Polling & Printing

Theory: Polls kernel maps periodically and formats histograms.

Details: The user-space printer queries maps, pulling bucket logs to scale and render text-based histograms (e.g. '[16, 32) 20 |@@@@@|') in the terminal.

Trade-off: Separates rendering overhead from the kernel, but polling very large map files can introduce latency spikes.

M3.5: Tracepoint Parsing

Theory: Extracts tracepoint arguments by parsing debugfs metadata formats.

Details: Parses system files (e.g., 'events/syscalls/sys_enter_openat/format') to compute argument offsets, enabling type-safe access like 'args->filename'.

Trade-off: Highly reliable but requires debugfs to be mounted on the host machine.

M4.1: Safe Mode Constraints

Theory: Restricts unsafe commands and system calls.

Details: Enforces constraints via '-safe' flag, blocking scripts from calling 'system()' or modifying file states.

Trade-off: Enhances security but limits reactive script automation scenarios.

M4.5: Signal Handling

Theory: Catches signals to safely detach hooks.

Details: Catches SIGINT/SIGTERM, flushing the remaining buffer before cleanly detaching all probes.

Trade-off: Crucial for system stability; prevents orphaned hook states.

M5.1: CLI Front End

Theory: Simplifies compiler invocation and target execution.

Details: The CLI frontend coordinates arguments parsing, JIT compilation, and execution cycles.

Trade-off: Streamlines debugging but lacks advanced multi-node orchestration capabilities.

M5.2: BTF Cache

Theory: Caches parsed BTF metadata to speed up compilation.

Details: Parses system BTF once, caching kernel type descriptions to accelerate subsequent compilation phases.

Trade-off: Memory-bound; caching metadata increases memory footprints during active compiler sessions.

M5.3: User-State State Tracking

Theory: Links user-defined script states across probe invocations.

Details: Tracks non-map local variables across probe points by routing updates through internal maps.

Trade-off: Map updates introduce minor lookup overhead compared to local registers.

M5.4: Multitenant Isolation

Theory: Isolates concurrently running tracing sessions.

Details: Leverages independent map file descriptors to prevent sessions from corrupting each other's metrics.

Trade-off: Requires elevated privileges ('CAP_BPF'), restricting usage to authorized system administrators.

Zone T1: bpftrace Core APIs

```
bpfftrace::BPFFtrace::add_probe(),      bpf-
trace::Driver::parse(),    bpfftrace::SemanticAnalyser::analyse(),
bpfftrace::CodegenLLVM::compile(),      bpf-
trace::attached_probe::attach(),        bpf-
trace::libbpf::bpf_map_update_elem(),    bpf-
trace::Printer::print_map()
```

Zone T2: Compilation & Execution Errors

```
ast_parse_error,      semantic_analysis_type_mismatch,
llvm_jit_codegen_failed,    ebpf_verifier_rejected_bytecode,
ring_buffer_overflow_dropped_events,
btf_vmlinux_not_found
```